



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
 ESCUELA DE INGENIERÍA
 DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2' 2025
 IIC2224 — Autómatas y Compiladores — 2' 2025

PAUTA TAREA 6

Pregunta 1

Sea $D = (Q, \Sigma, \Delta, q_0, F)$ un autómata apilador alternativo. Construiremos el autómata apilador alternativo $D' = (Q', \Sigma, \Delta', q'_0, F)$ tal que:

$$\begin{aligned} Q' &= Q \cup \{q'_0\} \cup \{1, 2, 3\} \\ \Delta' &= \{(q'_0, \epsilon, 1 \cdot q_0)\} \\ &\cup \{(1 \cdot \alpha, \epsilon, 1 \cdot \beta) \mid \exists c \in \Sigma \cup \{\epsilon\}. (\alpha, c, \beta) \in \Delta\} \\ &\cup \{(1, \epsilon, 2)\} \\ &\cup \{(2 \cdot \alpha, a, 2 \cdot \beta) \mid (\alpha, a, \beta) \in \Delta\} \\ &\cup \{(2, \epsilon, 3)\} \\ &\cup \{(3 \cdot \alpha, \epsilon, 3 \cdot \beta) \mid \exists c \in \Sigma \cup \{\epsilon\}. (\alpha, c, \beta) \in \Delta\} \\ &\cup \{(3 \cdot q_f, \epsilon, q_f) \mid q_f \in F\} \end{aligned}$$

Intuitivamente, D' distingue entre 3 fases:

1. Fase en que se simula a D leyendo el prefijo u sin consumir letras.
2. Fase en que se simula a D leyendo exactamente la parte v , que es la que queremos aceptar.
3. Fase en que se simula a D leyendo el sufijo w sin consumir letras.

Demostraremos que $\mathcal{L}(D') = \text{Mitad}(D)$.

$\mathcal{L}(D') \subseteq \text{Mitad}(D)$: Sea $v \in \mathcal{L}(D')$, entonces existe una ejecución de aceptación ρ de D' sobre v . Notemos que esta ejecución necesariamente debe partir en la fase 1, luego pasar por la fase 2, y finalmente llegar a la fase 3 para poder eliminar el marcador de fase y aceptar. Por lo tanto, la ejecución debe tener la siguiente forma:

$$\rho : (q'_0, v) \vdash_{D'} \underbrace{(1 \cdot q_0, v) \vdash_{D'}^* (1 \cdot \alpha, v) \vdash_{D'}}_u \underbrace{(2 \cdot \alpha, v) \vdash_{D'}^* (2 \cdot \beta, \epsilon) \vdash_{D'}}_v \underbrace{(3 \cdot \beta, \epsilon) \vdash_{D'}^* (3 \cdot q_f, \epsilon) \vdash_{D'}}_w (q_f, \epsilon)$$

Considerando que la ejecución utiliza las transiciones en Δ sólo añadiendo los marcadores de fase 1, 2 o 3, sabemos que existe la siguiente ejecución de aceptación de D sobre $u \cdot v \cdot w$:

$$\sigma : (q_0, u \cdot v \cdot w) \vdash_D^* (\alpha, v \cdot w) \vdash_D^* (\beta, w) \vdash_D^* (q_f, \epsilon)$$

Por lo tanto, $u \cdot v \cdot w \in \mathcal{L}(D)$ y entonces $v \in \text{Mitad}(D)$.

$Mitad(D) \subseteq \mathcal{L}(D')$: Sea $v \in Mitad(D)$, entonces existe un $u \in \Sigma^*$ y $w \in \Sigma^*$ tal que $u \cdot v \cdot w \in \mathcal{L}(D)$. Por lo tanto, existe la siguiente ejecución de aceptación de D sobre $u \cdot v \cdot w$:

$$\sigma : (q_0, u \cdot v \cdot w) \vdash_D^* (\alpha, v \cdot w) \vdash_D^* (\beta, w) \vdash_D^* (q_f, \epsilon)$$

Luego, por construcción de Δ' , existe la siguiente ejecución de aceptación de D' sobre v :

$$\rho : (q'_0, v) \vdash_{D'} \underbrace{(1 \cdot q_0, v) \vdash_{D'}^* (1 \cdot \alpha, v) \vdash_{D'}}_u \underbrace{(2 \cdot \alpha, v) \vdash_{D'}^* (2 \cdot \beta, \epsilon) \vdash_{D'}}_v \underbrace{(3 \cdot \beta, \epsilon) \vdash_{D'}^* (3 \cdot q_f, \epsilon) \vdash_{D'}}_w (q_f, \epsilon)$$

En consecuencia, $v \in \mathcal{L}(D')$.

Como $\mathcal{L}(D') \subseteq Mitad(D)$ y $Mitad(D) \subseteq \mathcal{L}(D')$, entonces queda demostrado que $\mathcal{L}(D') = Mitad(D)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por agregar estados que representan las fases de lectura.
- **(1 punto)** Por definir transiciones para pasar de una fase a otra.
- **(1 punto)** Por definir transiciones dentro de cada fase.
- **(1 punto)** Por demostrar correctitud.

Pregunta 2

Sea un autómata $A = (Q, \Sigma, \Delta, I, F)$. Sin pérdida de generalidad, supondremos que:

(*) para todo estado $q \in Q$, existe $w \in \Sigma^*$ tal que $\Delta^*(q, w) \cap F \neq \emptyset$

En otras palabras, la propiedad (*) establece que no hay estados *sumideros*: desde cualquier estado q , siempre existe una palabra que lleva desde q hasta algún estado final. En particular, podemos garantizar esta propiedad inicial porque, dado un autómata genérico A , es posible transformarlo en otro autómata equivalente A' (esto es, $\mathcal{L}(A) = \mathcal{L}(A')$) removiendo todos los estados que no son capaces de alcanzar un estado final. Esta operación no altera el lenguaje reconocido, ya que tales estados nunca participan en una ejecución de aceptación. Al quedarnos únicamente con los estados que sí pueden llegar a un estado final, garantizamos que todo estado no vacío tiene un camino hacia un estado final. Importante destacar que el computo de este paso pudo haberse realizado de distintas formas como un BFS inverso desde los estados finales o un DFS desde los estados individuales no superando una complejidad de $|Q| + |\Sigma|$ lo cual no altera la complejidad de la solución buscada. Por lo tanto, *sin pérdida de generalidad*, es válido asumir que el autómata resultante de realizar esta limpieza cuenta con transiciones a estados que son capaces de llegar a un estado final.

Con esto garantizado, la idea del algoritmo propuesto es realizar un backtracking de forma que solo vaya expandiendo estados que sabemos que son capaces de alcanzar un estado final. A medida que vayamos avanzando en estos devolver las palabras vistas hasta el momento.

Dado lo anterior, el algoritmo es el siguiente:

Input: Autómata $A = (Q, \Sigma, \Delta, I, F)$, conjunto de estados S , palabra $w \in \Sigma$ y un largo $j \in \mathbb{N}$.

Output: Imprimir todas las palabras wu con $|u| \leq j$ tal que $wuv \in \mathcal{L}(A)$ para algún $v \in \Sigma^*$.

Function First(A, S, w, j)

```

    print( $w$ )
    // Caso base: no se puede extender más
    if  $j = 0$  then
        return
    // Construcción de los conjuntos  $S_a$ 
    foreach  $a \in \Sigma$  do
         $S_a \leftarrow \{q \mid \exists p \in S : (p, a, q) \in \Delta\}$ 
    // Conjunto de extensiones válidas
     $A' \leftarrow \{(S_a, a) \mid S_a \neq \emptyset\}$ 
    // Llamada recursiva por cada transición válida
    foreach  $(S_a, a) \in A'$  do
        First( $A, S_a, wa, j - 1$ )

```

Algorithm 1: An algorithm with caption

Y para calcular $\text{first}_k(L(A))$ la llamada inicial a la función sería: $\text{First}(A, I, \epsilon, k)$

Correctitud: Para demostrar correctitud demostraremos la doble contención entre el lenguaje generado por el algoritmo y el lenguaje pedido:

Correctitud del algoritmo First

Queremos demostrar que el algoritmo

$\text{First}(A, S, w, j)$,

cuyo llamado inicial es $\mathbf{First}(A, I, \epsilon, k)$ produce exactamente el conjunto de palabras

$$\mathbf{first}_k(L(A)) = \{ u \in \Sigma^{\leq k} \mid \exists v \in \Sigma^*, uv \in L(A) \}$$

Para ello realizaremos una demostración por doble contención, es decir, probaremos que para toda palabra w se cumple:

$$w \text{ es impresa por } \mathbf{First} \iff w \in \mathbf{first}_k(L(A)).$$

Antes de proceder, recordemos que mediante un preprocesamiento estándar se eliminan del autómata todos los estados que no pueden alcanzar un estado final, operación que no altera el lenguaje reconocido. Por lo tanto, durante toda la ejecución del algoritmo cualquier estado presente en un conjunto S satisfará que existe un camino desde él hacia un estado final.

($\Rightarrow$) Si el algoritmo imprime w , entonces $w \in \mathbf{first}_k(L(A))$

Sea w una palabra que el algoritmo imprime en alguna llamada

$$\mathbf{First}(A, S, w, j).$$

Por construcción, dicha llamada sólo ocurre si $|w| \leq k$, pues el parámetro j comienza en k y se decrementa con cada extensión, deteniéndose cuando $j = 0$. Observemos que $S = \Delta(I, w)$, es decir, S corresponde al conjunto de estados alcanzables al leer w desde algún estado inicial. Dado el preprocesamiento, todo estado $p \in S$ puede alcanzar un estado final mediante alguna palabra $v \in \Sigma^*$. Por lo tanto:

$$\exists v \in \Sigma^* : \Delta(S, v) \cap F \neq \emptyset.$$

Equivalentemente,

$$\Delta(I, wv) \cap F \neq \emptyset.$$

Así, $wv \in L(A)$ y, puesto que $|w| \leq k$, concluimos que w es un prefijo de una palabra aceptada y de largo a lo más k . Es decir,

$$w \in \mathbf{first}_k(L(A)).$$

($\Leftarrow$) Si $w \in \mathbf{first}_k(L(A))$, entonces el algoritmo imprime w

Sea ahora $w \in \mathbf{first}_k(L(A))$. Por definición, existe $v \in \Sigma^*$ tal que $wv \in L(A)$ y $|w| \leq k$. Sea

$$w = a_1 a_2 \dots a_m \quad (m \leq k)$$

y consideremos una ejecución aceptante:

$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_m} q_m \xrightarrow{v} q_f,$$

donde $q_0 \in I$ y $q_f \in F$.

Procederemos por inducción en el largo m de la palabra w .

Caso base ($m = 0$). Aquí $w = \epsilon$. Como $\epsilon v \in L(A)$, los estados iniciales pertenecen al conjunto de estados válidos. El llamado inicial

$$\mathbf{First}(A, I, \epsilon, k)$$

imprime ϵ . Por lo tanto, el caso base es correcto.

Paso inductivo. Supongamos que toda palabra válida de largo m es impresa por el algoritmo. Sea ahora $w = ua$ con $|u| = m$ y supongamos que existe una ejecución aceptante que consume ua . Por hipótesis inductiva, el algoritmo imprime u al ejecutar

$$\text{First}(A, S_u, u, j),$$

donde $S_u = \Delta(I, u)$. En esta llamada, el algoritmo construye para cada símbolo $a \in \Sigma$ el conjunto

$$S_a = \{ q \mid \exists p \in S_u : (p, a, q) \in \Delta \}.$$

Como la ejecución aceptante contiene la transición $q_m \xrightarrow{a} q_{m+1}$, necesariamente $q_{m+1} \in S_a$, por lo que $S_a \neq \emptyset$. El algoritmo entonces incluye (S_a, a) en el conjunto A' y realiza la llamada recursiva:

$$\text{First}(A, S_a, ua, j - 1)$$

la que imprime $ua = w$. Así, toda palabra w con $|w| \leq k$ y que sea prefijo de una palabra aceptada será generada por el algoritmo.

Conclusión

Hemos demostrado ambas contenciones:

$$w \text{ es impresa por el algoritmo First} \iff w \in \text{first}_k(L(A)).$$

Por lo tanto, el algoritmo es correcto.

Complejidad

En cada llamada útil del algoritmo se construyen los conjuntos

$$S_a = \{ q \mid \exists p \in S : (p, a, q) \in \Delta \},$$

para todos los símbolos $a \in \Sigma$.

En el peor caso $S = Q$, por lo que revisar todas las transiciones posibles cuesta

$$O(|Q| \cdot |\Sigma|).$$

Por lo tanto, la complejidad total es

$$\boxed{O(|Q| \cdot |\Sigma| \cdot |\text{first}_k(L(A))|)}$$

más un preprocesamiento inicial $O(|Q| + |\Delta|)$, que se asumió como paso previo para depurar estados que no llegan a un estado final.

Distribución de puntaje:

- **(1 punto)** Por la noción de pre-procesamiento de estado "sumideros".
- **(1 punto)** Por la idea de implementación de un backtracking inteligente o algún tipo de búsqueda que evite expansiones innecesarias.
- **(1 punto)** Por el uso del on the fly para avanzar los estados.
- **(0.5 puntos)** Por la demostración de correctitud.
- **(0.5 puntos)** Por la demostración de complejidad.